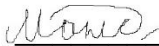


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

СОСТАВНЫЕ СПИСКИ

Отчет по практической работе по дисциплине «Основы интеллектуальных
систем»

Студент гр. 591-1

 Д.Ю. Мандриков

«28» мая 2024 г.

Руководитель

К.т.н., доцент каф ЭМИС

_____ Н.Ю. Истомина

оценка

«__» _____ 2024 г.

Томск 2024

Практическая работа №10

«Составные списки»

Цель работы: освоить основные приемы работы со списками в языке Пролог.

Краткие теоретические сведения.

Составные домены и списки создаются с использованием встроенных (built-in) и других составных или списковых доменов. Справочная система Visual Prolog (Help) объясняет встроенные домены:

- integer – целые;
- real – вещественные;
- string – строковые;
- symbol –символьные;
- char – знаковые;
- string8 - строковые 8-разрядные;
- pointer – указатели;
- binary - двоичные (бинарные);
- boolean – булевские;
- object – объекты.

Составные домены позволяют рассматривать наборы данных как единое целое и при этом мы можете рассматривать их и отдельно. Рассмотрим, например, дату "October 15, 2003". Она состоит из трех единиц 76 информации – месяца, дня и года – но было бы полезно рассматривать дату как единое целое в виде древовидной структуры. Это выглядит как факт Пролога, но это не так – это всего лишь значение, которое можно обрабатывать почти также, как строки или числа. Такая структура начинается с имени, обычно называемым функтор (functor) (в данном случае - date), за которым следуют три аргумента. Обращаем Ваше внимание на то, что функтор в Visual Prolog не имеет ничего общего с функциями в языках программирования. Функтор не вызывает никаких вычислений, это всего лишь имя, которое

идентифицирует составную величину и объединяет свои аргументы. Аргументы составной величины могут быть, в свою очередь, составными.

Значения составных доменов могут унифицироваться либо с одиночными переменными, либо с составными значениями, которые сопоставимы с ними (здесь возможны переменные, как части внутренней структуры). Это значит, что Вы можете использовать составные величины для передачи в виде целого набора данных, а затем разбирать их по частям путем операции унификации.

Visual Prolog выполняет унификацию в двух случаях. Первый - это когда вызов сопоставляется с головой клаузы. Второй - это при использовании знака равно (=), который в действительности является инфиксной формой предиката (предикат, который находится между своими аргументами, вместо того, чтобы быть перед ними). Visual Prolog осуществляет необходимые связывания для унификации значений по обе стороны от знака равно. Это полезно для выделения значений аргументов в составных величинах. Например, следующий код проверяет имеют ли два человека одинаковы фамилии, после чего второй человек получает тот же адрес, что и первый.

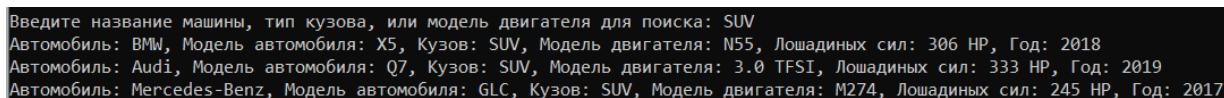
Важным свойством составных доменов является возможность легкой передачи сгруппированных данных с помощью одного аргумента. Рассмотрим случай поддержки базы данных телефонных номеров. В эту базу данных мы хотим включать также дни рождения своих друзей и членов их семей. Анализируя данные, замечаем, что факт `phone_list` имеет шесть аргументов; пять из них могут разбиты на два составных домена. Было бы более полезно представить эти факты в виде составных доменов. Отступив на шаг, видим, что персону (`person`) является отношением, а имя (`First Name`) и фамилия (`Last Name`) являются его аргументами. Кроме того, день рождения является также отношением с тремя аргументами: месяц (`month`), день (`day`) и год (`year`). Представление этих отношений средствами Пролога аналогично уже упоминавшемуся. Предикат `phone_list` теперь содержит три аргумента вместо прежних шести. Иногда разбиение данных на составные структуры упрощает

программу и облегчает обработку данных. Теперь добавим некоторые правила в программу. Предположим, нам надо получить список людей, чьи дни рождения выпадают на текущий месяц. Ниже приведена программа, решающая эту задачу; эта программа использует стандартный предикат `date` для получения текущей даты из внутренних часов компьютера. Предикат `date` возвращает текущий год (`year`), месяц (`month`) и день (`day`) часов компьютера.

Формулировки заданий с полученными результатами.

Для составных списков была выбрана предметная область «Машина». Для реализации было определен список `car`, который состоит из 3 элементов `names`, `BodyType`, `engine`. `Names` список, который содержит название и модель машины. `BodyType` строка, которая обозначает тип кузова машины. `Engine` список, который содержит данные о моторе: модель, лошадиные силы и год.

Реализован поиск строки по вхождению в список. Результат работы программы представлен на рисунке 1.



```
Введите название машины, тип кузова, или модель двигателя для поиска: SUV
Автомобиль: BMW, Модель автомобиля: X5, Кузов: SUV, Модель двигателя: N55, Лошадиных сил: 306 HP, Год: 2018
Автомобиль: Audi, Модель автомобиля: Q7, Кузов: SUV, Модель двигателя: 3.0 TFSI, Лошадиных сил: 333 HP, Год: 2019
Автомобиль: Mercedes-Benz, Модель автомобиля: GLC, Кузов: SUV, Модель двигателя: M274, Лошадиных сил: 245 HP, Год: 2017
```

Рисунок 1 – Скриншот результата работы программы

Код программы представлен в приложении А.

Вывод к практической работе: освоены основные приемы работы со списками в языке Пролог.

Приложение А

(обязательное)

class my

domains

car = car(string names, string BodyType, engine).

engine = engine(string Model, integer Horsepower, integer Year).

names = names(string NameCar, string ModelCar).

predicates

run : ().

checkCar : (car*).

end class

implement my

open core, console

clauses

checkCar(Cars) :-

 stdio::write("Введите название машины, тип кузова, или модель двигател
я для поиска: "),

 SearchQuery = console::readLine(),

 foreach car(names(NameCar, ModelCar), BodyType, engine(Model, Horsepo
wer, Year)) in Cars do

 if NameCar = SearchQuery or BodyType = SearchQuery or Model = Searc
hQuery then

 stdio::write("Автомобиль: ", NameCar, ", Модель автомобиля: ", Mod
elCar, ", Кузов: ", BodyType, ", Модель двигателя: ", Model, ", Лошадиных сил
: ", Horsepower, " HP, Год: ", Year, "\n")

```
end if
end foreach.
```

```
run() :-
  console::init(),
  Cars =
    [
      car(names("BMW", "X5"), "SUV", engine("N55", 306, 2018)),
      car(names("Audi", "Q7"), "SUV", engine("3.0 TFSI", 333, 2019)),
      car(names("Mercedes-
Benz", "GLC"), "SUV", engine("M274", 245, 2017)),
      car(names("Toyota", "Camry"), "Sedan", engine("2AR-FE", 181, 2016)),
      car(names("Honda", "Accord"), "Sedan", engine("K24W", 185, 2015)),
      car(names("Ford", "Focus"), "Hatchback", engine("EcoBoost", 150, 201
7)),
      car(names("Chevrolet", "Cruze"), "Hatchback", engine("LUJ", 140, 2016
)),
      car(names("Volkswagen", "Golf"), "Hatchback", engine("TSI", 110, 201
5)),
      car(names("Tesla", "Model S"), "Sedan", engine("Electric", 421, 2020)),
      car(names("Nissan", "Leaf"), "Hatchback", engine("Electric", 150, 2019)
)
    ],
  checkCar(Cars).
```

```
end implement my
```

```
goal
  console::runUtf8(my::run).
```